



**RV College of
Engineering®**

Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Go, change the world

Automated Fruit Freshness Detection for Inventory Management

Team No:3

USN	Name
1RV23AI018	Anjali Suresh Kalarikkal
1RV23AI020	Apoorva Krishna P
1RV23AI012	Alroy Deon Saldanha
1RV23AI017	Aniket RT



Agenda

- Introduction
- Problem Definition
- Objectives
- Data set Description
- Risk Analysis
- Tools and Techniques used
- Phase I : Machine Learning Model and its analysis
- Phase II : Building MLOPs



Introduction and Problem Definition

Large quantities of fruits are lost due to:

- Delayed quality assessment
- Inaccurate manual inspection

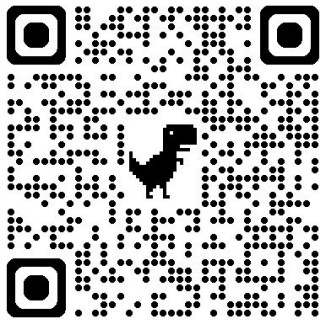
Motivation behind this project:

- Reduce post-harvest losses
- Improve warehouse operational efficiency
- Deploy a production-ready, scalable, and transparent AI system



Business Objectives

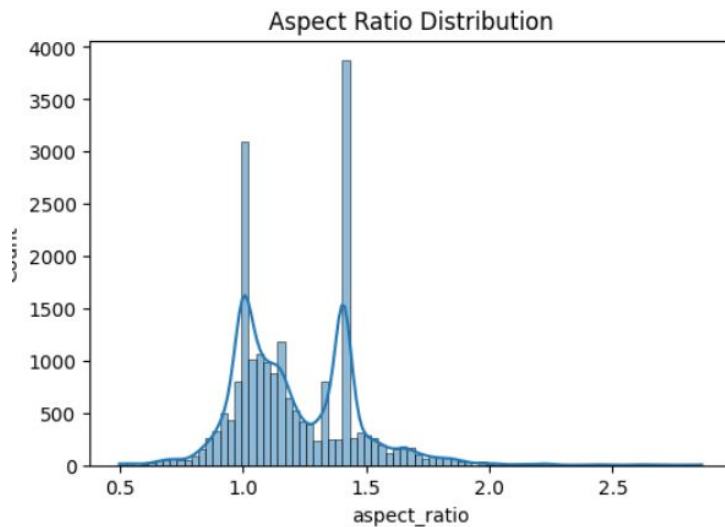
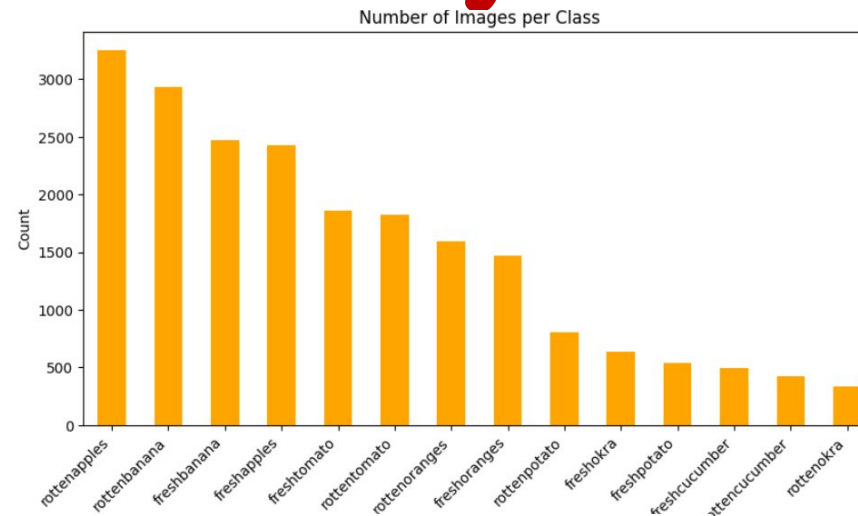
- 1. Develop a CNN-based fruit freshness detection model.**
- 2. Implement automated fruit removal by triggering mechanical or control actions when rotten fruits are detected.**
- 3. Feature drift detection and transparency through explainability**
- 4. Adherence to MLOps lifecycle for development and deployment.**



Exploratory Data Analysis

Metadata preview:

	class	filename	width	height	mode
0	freshapples	saltandpepper_Screen Shot 2018-06-08 at 5.18.2...	364	402	RGB
1	freshapples	a_f170.png	382	386	RGB
2	freshapples	a_f485.png	414	372	RGB
3	freshapples	vertical_flip_Screen Shot 2018-06-08 at 5.15.5...	448	426	RGB
4	freshapples	a_f259.png	400	380	RGB

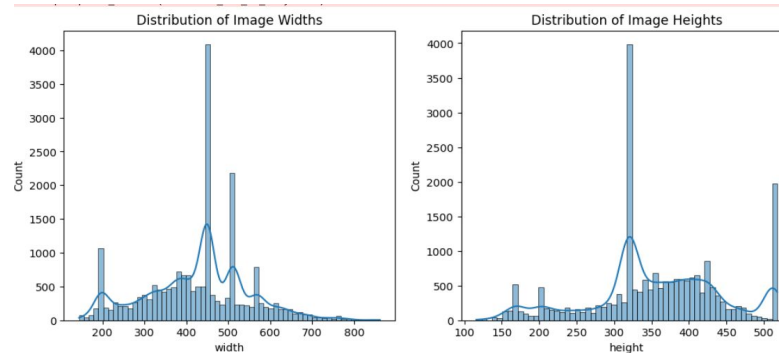


Corrupted or unreadable images: 0

Computing RGB stats: 100% | 14/14 [00:07<00:00, 1.89it/s]

Mean RGB values: [192.09506577 177.95038863 160.90092081]

Standard Deviation RGB values: [79.6499661 82.54764903 90.57108123]





Data Versioning - DVC

```
cat archive/dataset.dvc
```

```
- md5: fd7279d498bcd7c1e35cdd7979fdd541.dir
  size: 3033899151
  nfiles: 27782
  hash: md5
  path: dataset
```

```
python scripts/analyze_dataset.py
```

```
=====
Dataset Analysis Report
=====

Dataset Path: archive\dataset

Overall Statistics:
  Total Images: 27781

Quality Distribution:
  Fresh: 13127
  Rotten: 14654

Split Breakdown:
  Test: 6737 images (24.25%)
  Train: 21044 images (75.75%)
```

```
python scripts/validate_dataset.py
```

```
=====
Dataset Validation Report
=====

Status: PASSED
Dataset Path: archive\dataset

Summary:
  Total Files: 27781
  Total Categories: 28
  Train Split: 21044 images
  Test Split: 6737 images

Report saved to: data_validation_report.json
```

```
.\dvc_quickstart.ps1 - interactive menu
```

```
=====
DVC Dataset Management - Quick Start
=====

1. Validate dataset
2. Analyze dataset
3. Run full pipeline (validate + analyze)
4. Check DVC status
5. Push data to remote
6. Pull data from remote
7. View dataset info
8. Commit changes to Git
9. Exit

Enter your choice (1-9): : █
```



Experiment Tracking - MLflow

- An experiment named `manual_image_classification_experiment` was created.
- Each training run is tracked independently with a unique run ID.



mlflow 3.8.0

Manual image classification experiment > Runs > cnn_image_classifier

Overview | Model metrics | System metrics | Traces | Artifacts

Description

No description

Metrics (6)

Metric	Value	Models
final_train_accuracy	0.8212317228317261	cnn_model
final_val_accuracy	0.8155239224433899	cnn_model
train_accuracy	0.8212317228317261	cnn_model
train_loss	0.46653857827186584	cnn_model
val_accuracy	0.8155239224433899	cnn_model
val_loss	0.45926326513290405	cnn_model

About this run

Created at: 12/09/2025, 09:24:57 AM

Created by: root

Experiment ID: 617989921502070804

Status: Finished

Run ID: a609e5cf711949fb3e84f020a9cead6

Duration: 14.3min

Source: colab_kernel_launcher.py

Registered prompts: --

Datasets

None

Manual image classification experiment > Runs > cnn_image_classifier

Overview | Model metrics | System metrics | Traces | Artifacts

Search parameters

Parameter	Value
batch_size	32
epochs	10
image_size	150x150
loss	categorical_crossentropy
num_classes	14
optimizer	adam

Registered models

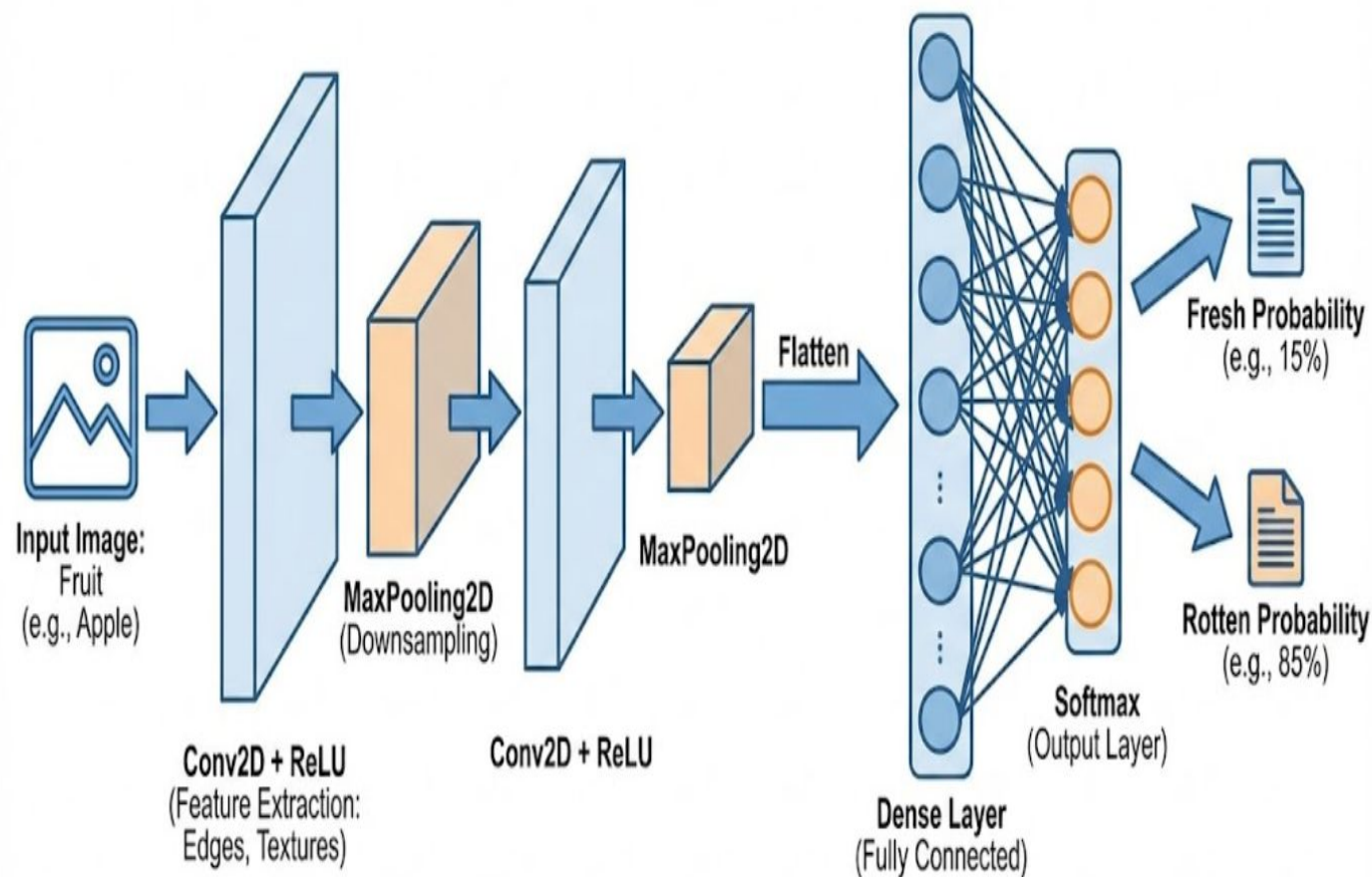
None

Logged models (1)

Type	Step	Model name	Status	Created	Registered models
Output	0	cnn_model	Ready	18 days ago	-



Models - Baseline CNN with fully connected layer



- **Input Stage:** The process begins with a raw image of the fruit (e.g., an apple), which is resized and normalized before entering the network.
- **Feature Extraction:** The image passes through three blocks of **Convolutional (Conv2D)** and **MaxPooling** layers. These layers act as automated feature detectors, starting with 32 filters to catch simple edges and scaling up to 128 filters to identify complex patterns like mold texture or skin discoloration.
- **Classification:** The learned features are flattened into a single vector and processed by fully connected **Dense Layers**. This stage weighs the features to make a final decision.
- **Output:** The final **Softmax layer** produces a probability distribution across the 14 classes, giving a confidence score (e.g., "98% Rotten Apple") to trigger the sorting mechanism.



Models

1) Custom CNN (Baseline)

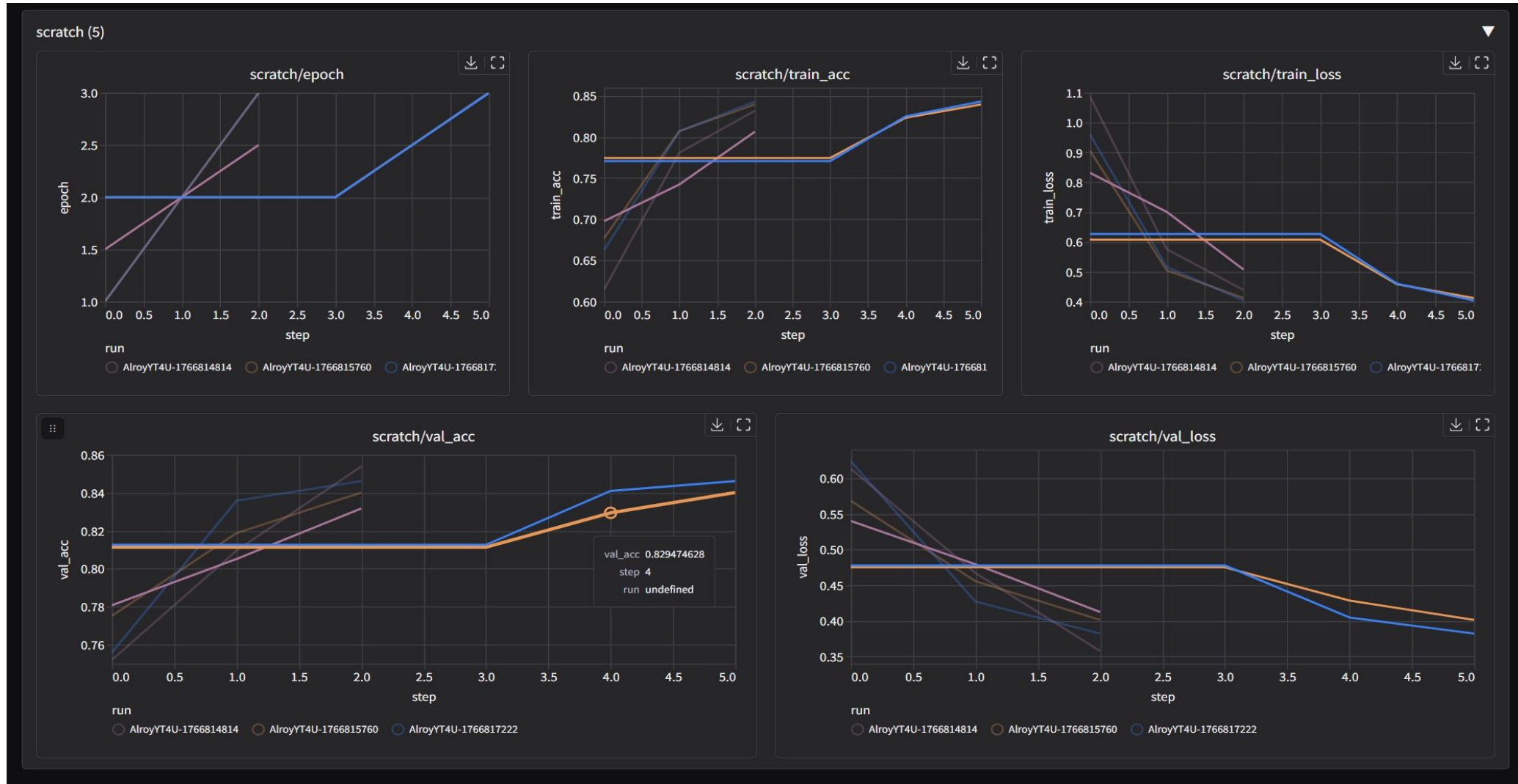
- **Architecture:** Built from scratch using a Sequential design with convolutional layers (32-128 filters) to extract specific fruit textures and edges.
- **Design:** A lightweight model (~18 MB) tailored specifically for the 14-class dataset to ensure low-latency inference on edge devices.
- **Performance:** Achieved ~84% accuracy, demonstrating capability but requiring more training time to learn features from scratch.

2) MobileNetV2 (Transfer Learning)

- **Architecture:** Leverages a pre-trained model (trained on ImageNet) to utilize powerful, pre-learned feature extractors for complex visual patterns.
- **Design:** Adapted via Transfer Learning, where the final classification layers were fine-tuned specifically for fruit freshness detection.
- **Performance:** Outperformed the baseline with ~95% accuracy and rapid convergence (reaching 92% in Epoch 1), making it the superior choice for production.

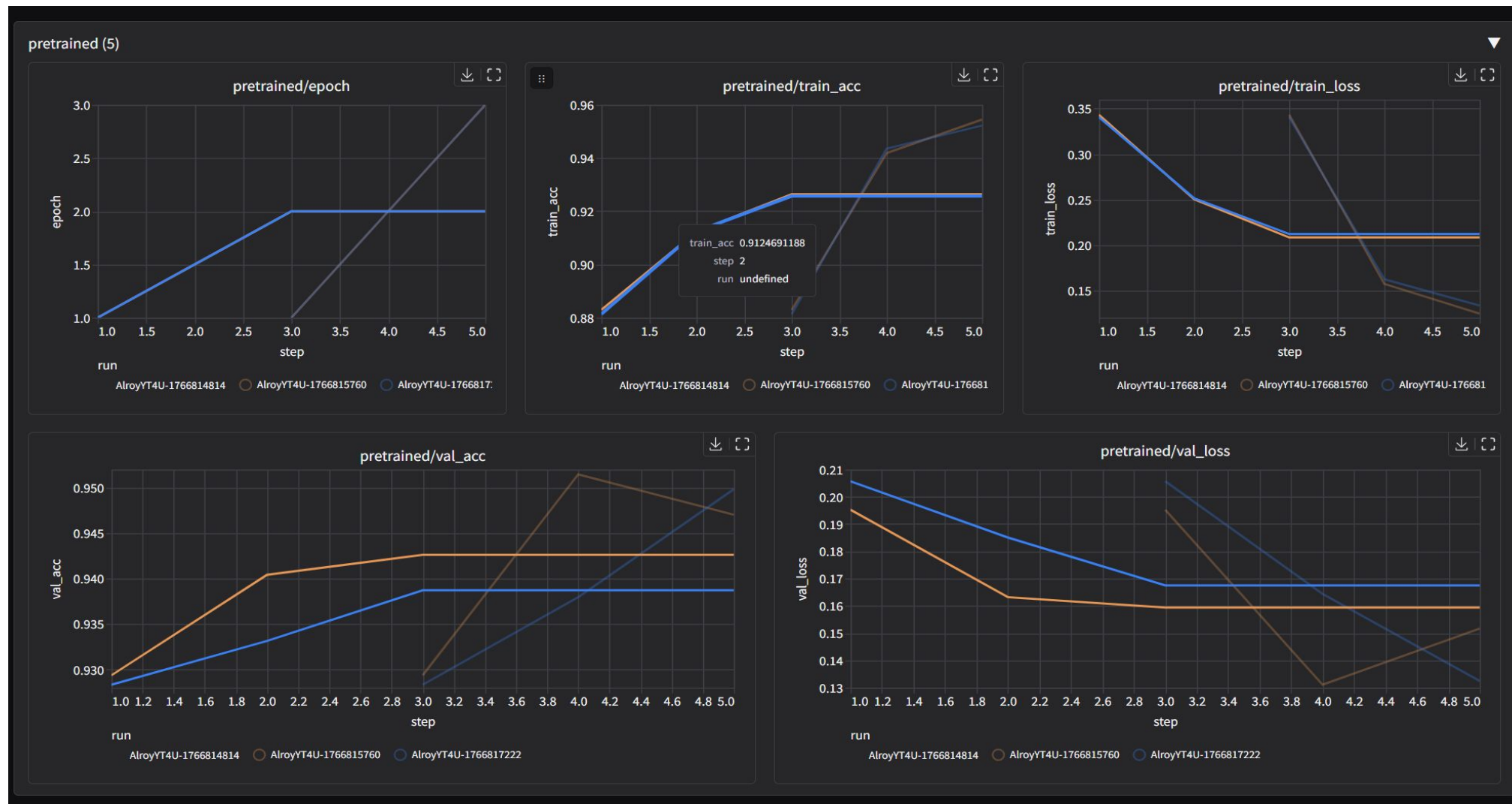
Monitoring and Logging

Scratch CNN



Monitoring and Logging

MobileNetV2



Monitoring and Logging

Persistent Data Storage (Hugging Face Datasets)

A critical component of this MLOps pipeline is the use of **Hugging Face Datasets** as a backend for log storage, ensuring **Data Lineage** and **Auditability**.

Unlike temporary in-memory logs, every metric recorded during training is structured and committed to a persistent dataset repository (as shown in the **Dataset Viewer**).

Dataset Viewer Auto-converted to Parquet </> API Embed Duplicate Data Studio

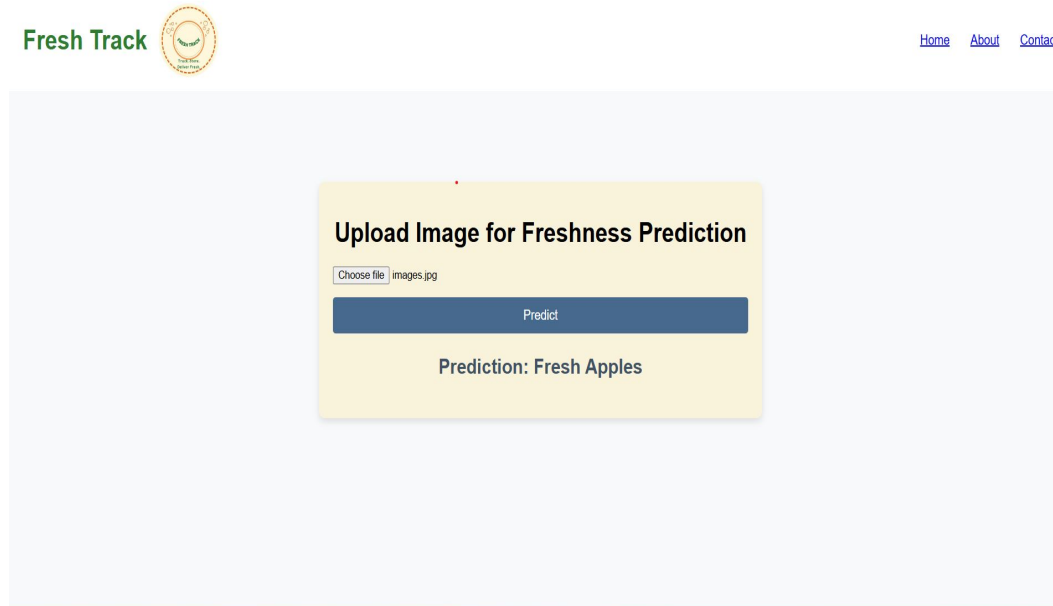
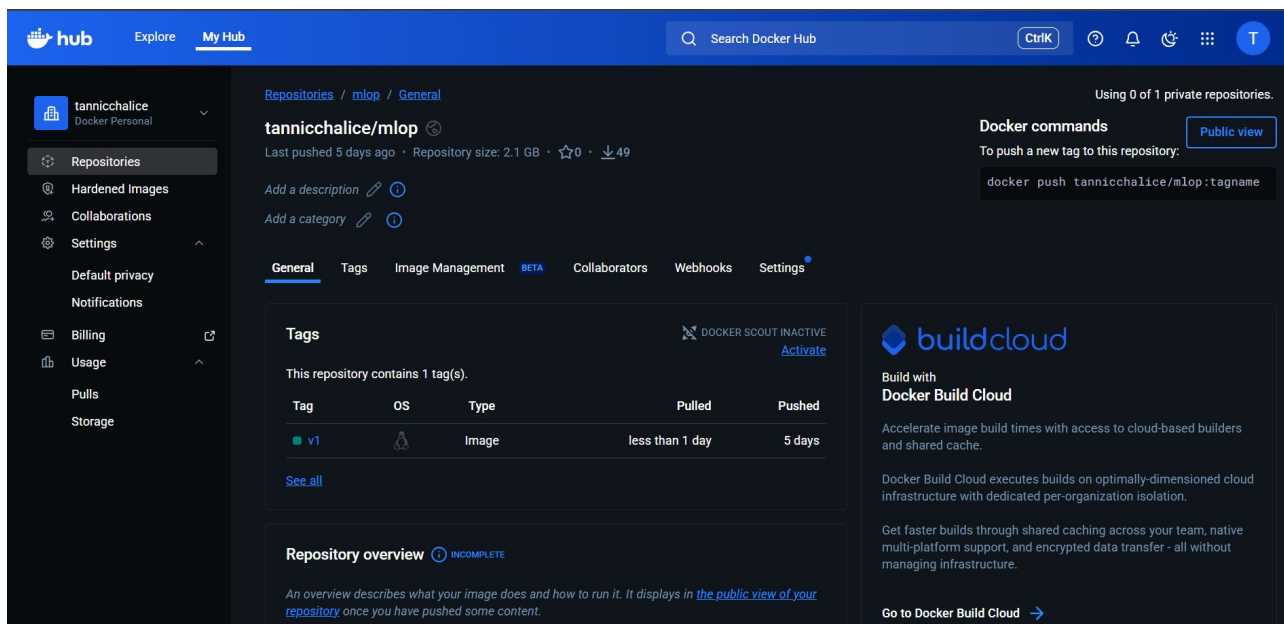
Split (1)
train · 15 rows

Search this dataset

timestamp string · date	run_name string · classes	step int64	scratch/epoch float64	scratch/train_loss float64	scratch/train_acc float64	scratch/val_acc float64
2025-12-27T06:57:27.403139	AlroyYT4U-1766814814	0	1	1.088008	0.614142	0.36
2025-12-27T07:00:49.845963	AlroyYT4U-1766814814	1	2	0.575367	0.780935	
2025-12-27T07:04:09.850372	AlroyYT4U-1766814814	2	3	0.438777	0.831734	
2025-12-27T07:12:55.918190	AlroyYT4U-1766815760	0	1	0.905988	0.67663	
2025-12-27T07:16:23.443984	AlroyYT4U-1766815760	1	2	0.50445	0.807261	
2025-12-27T07:19:46.398972	AlroyYT4U-1766815760	2	3	0.41142	0.839479	

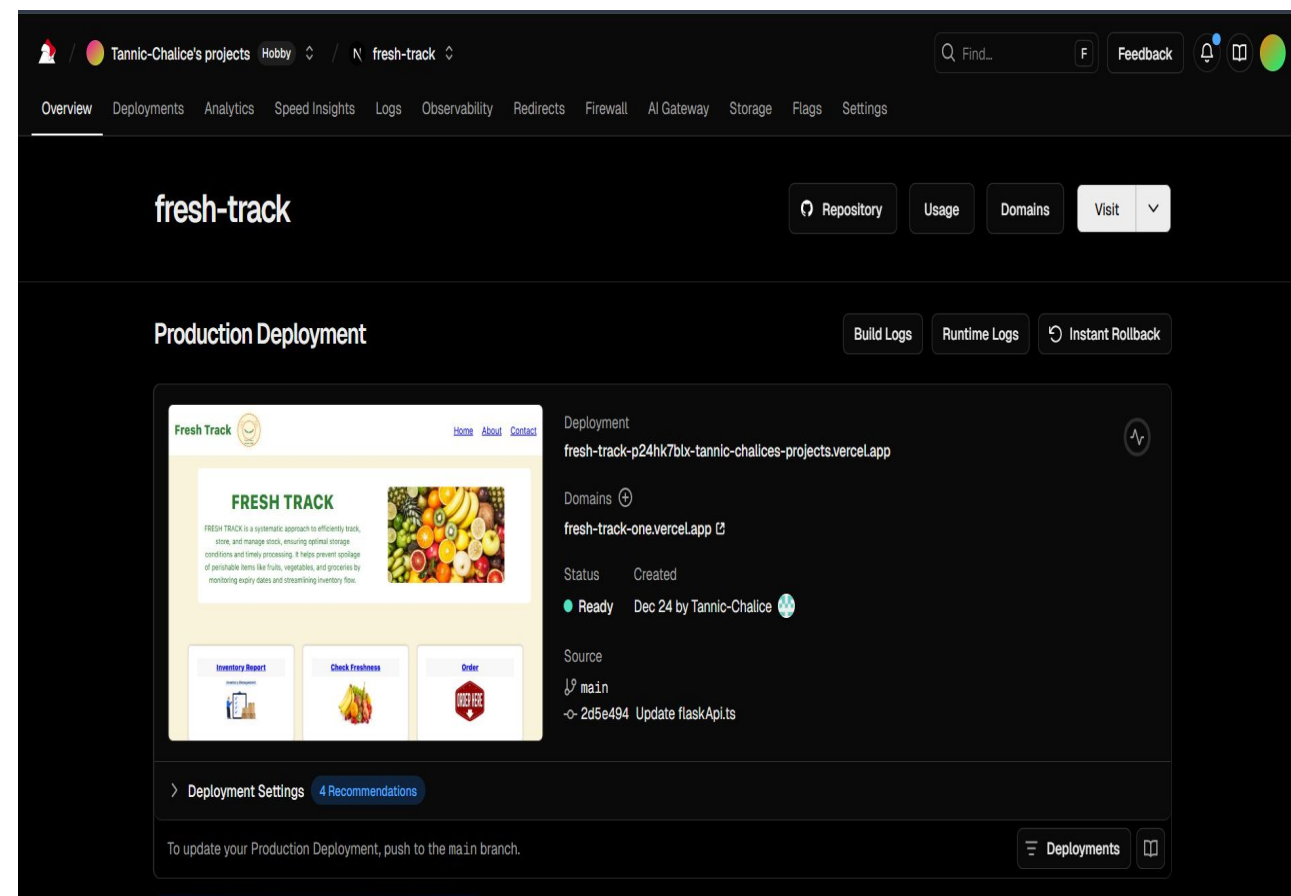
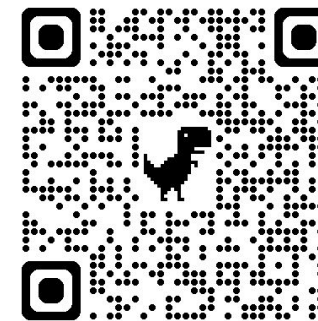
Containerization and CI/CD

- Once built, the Docker image is deployed on GCP, where it runs as a containerized service
- Sensitive configuration details, such as database credentials, are securely managed using environment variables injected at runtime, enhancing security and maintainability.





- Vercel is used to deploy NextJs framework.
- Any change pushed to the configured branch in the repository automatically triggers a build process followed by deployment.
- The build step performed by Vercel validates dependency resolution and successful compilation before deployment..





- **GCP is used to deploy our docker container.**
- **It handles all the business logic and API requests.**
- **The backend processes user actions such as adding, updating, and retrieving product details and ensures proper validation before interacting with the database.**

The screenshot displays the Google Cloud Console interface for a project named 'My First Project'. The 'Cloud Run' service is selected, showing details for a service named 'mlop' in the 'asia-south1' region. The service is currently deploying a revision. The 'Revisions' tab is active, showing a table of revisions. The first revision, 'mlop-00002-464', is the latest and has 100% traffic. The second revision, 'mlop-00001-6zk', is older and has 0% traffic. The right-hand panel shows the 'General' configuration for the selected revision, including billing, startup CPU boost, concurrency, request timeout, and execution environment.

Name	Traffic	Deployed	Revision	Actions
mlop-00002-464	100% (to latest)	3 minutes ago	+	⋮
mlop-00001-6zk	0%	20 minutes ago		⋮

General	
Billing	Request-based
Startup CPU boost	Enabled
Concurrency	80
Request timeout	300 seconds
Execution environment	Default



AWS RDS

- AWS RDS is used store the database.
- The database layer is implemented using PostgreSQL hosted on AWS RDS, where structured product data—including product ID, product name, quantity, price, and expiry date—is securely stored.
- This separation of concerns improves scalability, security, and maintainability of the application.



AWS RDS

```
postgres=> select * from grocery;
 id | Item_name | Category | Quantity | price | Supplier | Expiry_date
----+-----+-----+-----+-----+-----+-----
  3 | Rotten Banana | Fruits | 1 | 20 | Farm | 2025-12-30
  4 | Fresh Okra | Fruits | 1 | 20 | Farm | 2025-12-30
  6 | apples | fruits | 10 | 200 | abc | 2025-12-15
  7 | apples | fruits | 10 | 200 | abc | 2025-12-15
  8 | apples | fruits | 10 | 200 | abc | 2025-12-15
  2 | Fresh Apples | Fruits | 19 | 20 | Farm | 2026-01-02
  5 | apples | fruits | 8 | 200 | abc | 2025-12-15
  1 | Fresh Apples | Fruits | 27 | 20 | Farm | 2026-01-03
(8 rows)
```



Drift Detection

Types of Drift Monitored:

Feature Drift

- Detected on CNN feature embeddings
- Indicates the model is seeing new visual patterns

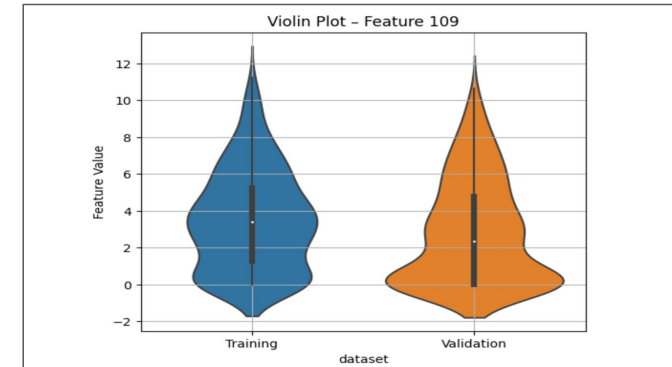
Prediction & Confidence Drift

- Detected on predicted class distribution and prediction
- confidence values

How Drift Was Detected:

1. Extracted baseline embeddings from trained CNN
2. Used Alibi Detect (KSDrift) to compare:
 - Training data (reference)
 - Validation / incoming data (current)
3. Drift flagged when statistical difference is significant ($p < 0.05$)

Feature Drift Plot



Confidence Output screenshot

```
=====
BASELINE CONFIDENCE STATISTICS
=====
Mean confidence: 0.8415
Std confidence: 0.1915
Min confidence: 0.1578
Max confidence: 1.0000

=====
TOP 3 PREDICTED CLASSES
=====
1. rottenapples: 15.88% (3338 samples)
2. rottenbanana: 13.12% (2758 samples)
3. freshbanana: 11.79% (2478 samples)
```

Model explainability

Method Used: Grad-CAM

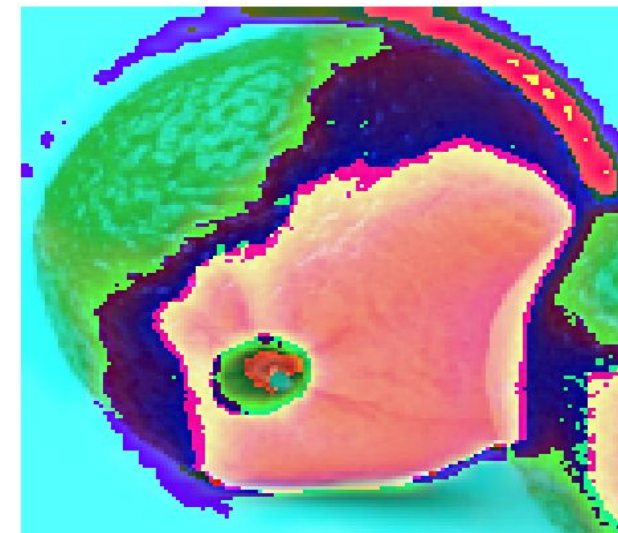
- Visualizes CNN attention by highlighting influential regions
- Validates statistically detected feature drift justifying the need for retraining

How explainability helps:

- Understand the model's decision
- Analyze model behavior under detected drift

Key Observations (Before Retraining)

- Partial attention on irrelevant areas
- Attention appears scattered, indicating unstable feature understanding





MLOPs Life Cycle

	Component	MLOPs Tools
1	Data Management	DVC
2	Experiment Tracking and Version Control	Mlflow
3	Model Explainability	GRAD-CAM
4	Model Deployment & Serving	Google Cloud for backend Vercel for frontend
5	Monitoring & Governance and collaboration	HuggingFace Spaces for Monitoring and Logging Github and Kaggle for collaboration
6	Drift Detection	Alibi detect
7	Containerization	Docker



Roles in Mlops life cycle

	Component	MLOPs Tools
1	Anjali	Data engineer and SWE
2	Apoorva	Model risk manager
3	Aniket	DevOps, SME
4	Alroy	Data Scientist and ML Architect



**RV College of
Engineering®**

Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Go, change the world

Thank You!